Ejercicio práctico: amplía la tienda

Paso a paso, con huecos para tus pantallazos

1. Qué vas a hacer

En este ejercicio vas a trabajar con el proyecto **tienda** que ya conoces. Primero lo pondrás en marcha tal como está y después le harás cuatro cambios pequeños:

- Añadir un pedido nuevo a la clienta **Ana**: una **Mochila**.
- Crear un cliente nuevo llamado **Luis** con dos pedidos: **Cuaderno** y **Goma**.
- Hacer que el programa busque a **Luis** en lugar de a Ana.
- Cambiar el título de la página web a **Clientes de mi tienda**.

Al terminar, la web mostrará a los dos clientes con todos sus pedidos.

Después de cada paso importante hay un recuadro como este para que pegues un pantallazo de lo que ves en tu ordenador:

> Para hacer un pantallazo en Windows pulsa a la vez las teclas **Windows + Mayús + S**, selecciona con el ratón la zona de la pantalla y después haz clic dentro del recuadro de este documento y pulsa **Ctrl + V** para pegarlo.

Los archivos del proyecto

Estos son los archivos con los que vas a trabajar y lo que hace cada uno:

Archivo	Qué hace
`modelos.py`	Tiene las clases **Cliente** y **Pedido**. No lo vas a tocar.
`mapeo.py`	El fichero de mapeo: une cada clase con su tabla. No lo vas a tocar.
`principal.py`	Crea la base de datos y guarda los clientes. **Aquí harás tres cambios.**
`app.py`	Arranca la página web. No lo vas a tocar.
`templates/index.html`	La página web que se ve en el navegador. **Aquí harás un cambio.**

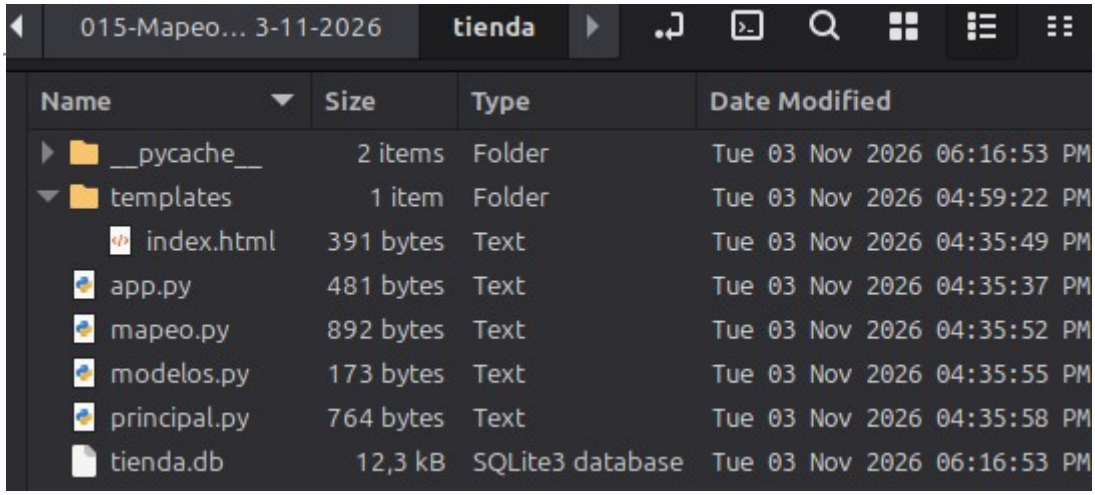
Solo vas a modificar **dos archivos**: `principal.py` e `index.html`.

2. Parte 1: poner en marcha el proyecto

Antes de cambiar nada, comprueba que el proyecto funciona tal como está.

Paso 1. Abrir la carpeta del proyecto

1. Abre el **Explorador de archivos** (el icono de la carpeta amarilla en la barra de tareas).
2. Entra en la carpeta **tienda** (haz doble clic hasta ver los archivos `.py`).
3. Comprueba que están los archivos `modelos.py`, `mapeo.py`, `principal.py`, `app.py` y la carpeta `templates`.
4. Haz doble clic en la carpeta `templates` y comprueba que dentro está `index.html`. Después vuelve atrás a la carpeta **tienda** con la flecha de la izquierda.



Paso 2. Abrir una terminal dentro de la carpeta

1. Estando dentro de la carpeta **tienda**, haz clic en la **barra de direcciones** (arriba del todo, donde aparece la ruta de la carpeta). El texto se quedará seleccionado.
2. Borra lo que pone, escribe `cmd` y pulsa **Enter**.
3. Se abrirá una ventana negra (el Símbolo del sistema). La última línea termina en `tienda>`. Eso significa que la terminal está trabajando dentro de la carpeta **tienda**.

No cierres esta ventana: la vas a usar durante todo el ejercicio.



PANTALLAZO 2

Pega aquí: la terminal abierta, donde se vea que la ruta termina en tienda>.

Paso 3. Instalar las librerías

Si ya las instalaste en otra ocasión, puedes saltarte este paso. Si no, escribe en la terminal esta orden y pulsa **Enter**:

```
pip install sqlalchemy flask
```

Espera a que termine. Al final aparecerá **Successfully installed** (instalado correctamente) o **Requirement already satisfied** (ya estaba instalado). Las dos respuestas son buenas.

```
cicios/015-Mapeo Python 3-11-2026/tienda$ pip install sqlalchemy flask --break-s
ystem-packages
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: sqlalchemy in /home/hiro/.local/lib/python3.12/si
te-packages (2.0.35)
Requirement already satisfied: flask in /usr/lib/python3/dist-packages (3.0.2)
Requirement already satisfied: typing-extensions>=4.6.0 in /home/hiro/.local/lib
/python3.12/site-packages (from sqlalchemy) (4.15.0)
Requirement already satisfied: greenlet!=0.4.17 in /home/hiro/.local/lib/python3
.12/site-packages (from sqlalchemy) (3.3.1)
Requirement already satisfied: Werkzeug>=3.0.0 in /usr/lib/python3/dist-packages
 (from flask) (3.0.1)
Requirement already satisfied: Jinja2>=3.1.2 in /usr/lib/python3/dist-packages (
from flask) (3.1.2)
Requirement already satisfied: itsdangerous>=2.1.2 in /usr/lib/python3/dist-pack
ages (from flask) (2.1.2)
Requirement already satisfied: click>=8.1.3 in /usr/lib/python3/dist-packages (f
rom flask) (8.1.6)
Requirement already satisfied: blinker>=1.6.2 in /usr/lib/python3/dist-packages
(from flask) (1.7.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/lib/python3/dist-packag
es (from Werkzeug>=3.0.0->flask) (2.1.5)
```

PANTALLAZO 3

Pega aquí: la terminal con el final de la instalación.

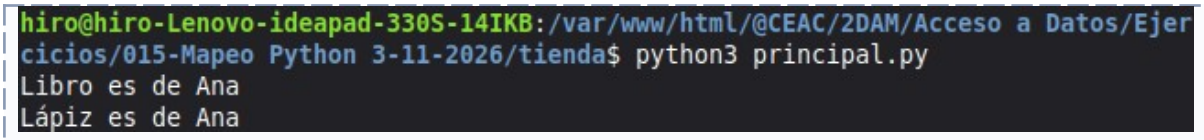
Paso 4. Crear los datos

Escribe en la terminal esta orden y pulsa **Enter**:

```
python principal.py
```

Este programa crea la base de datos `tienda.db`, guarda a la clienta Ana con sus pedidos y después la busca para mostrarlos. Debe aparecer:

```
Libro es de Ana
Lápiz es de Ana
```



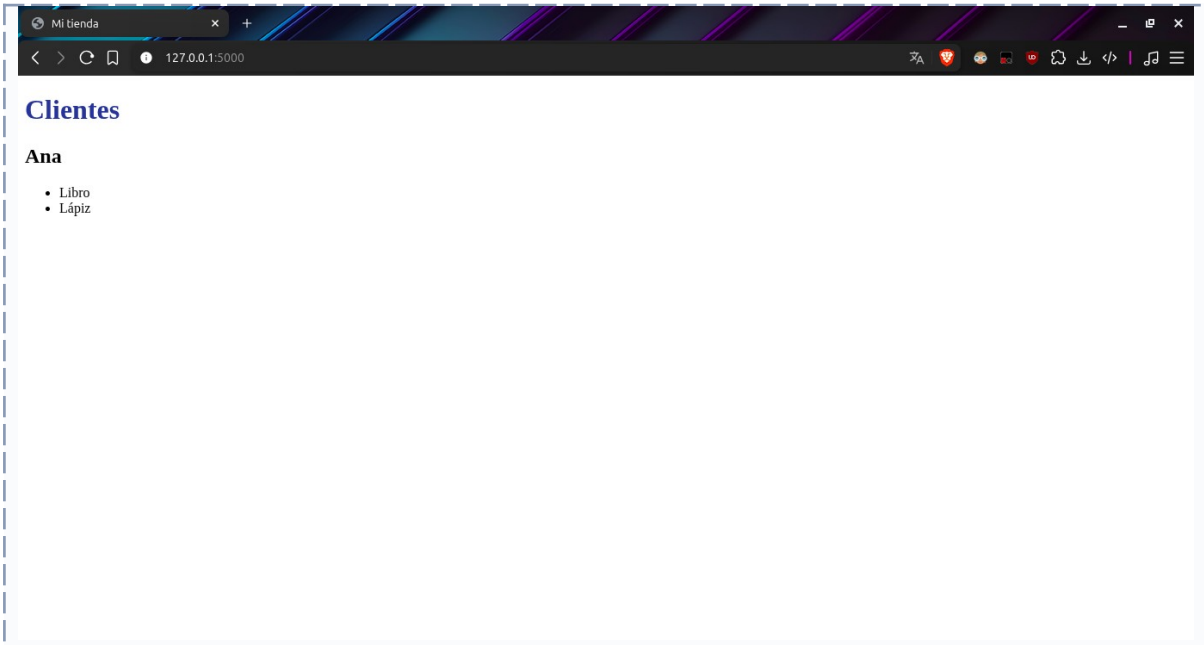
PANTALLAZO 4

Pega aquí: la terminal mostrando las dos líneas anteriores.

Paso 5. Ver la página web

1. Escribe en la terminal `python app.py` y pulsa **Enter**.
2. Espera a que aparezca una línea parecida a `* Running on http://127.0.0.1:5000`. Significa que la web está funcionando.
3. Abre el navegador (Chrome, Edge o Firefox).
4. En la barra de direcciones del navegador escribe `http://127.0.0.1:5000` y pulsa **Enter**.

5. Verás el título **Clientes** y, debajo, a **Ana** con sus pedidos **Libro** y **Lápiz**.



PANTALLAZO 5

Pega aquí: el navegador mostrando la web con Ana y sus dos pedidos.

Paso 6. Parar el servidor

Para poder seguir escribiendo órdenes hay que parar el servidor web:

1. Haz clic dentro de la ventana negra de la terminal.
2. Pulsa a la vez las teclas **Ctrl + C**.
3. Vuelve a aparecer la línea que termina en `tienda>`. El servidor está parado.

> Haz esto cada vez que termines de mirar la web. Mientras el servidor está funcionando, la terminal no acepta otras órdenes.

3. Parte 2: añadir una mochila a Ana

Ahora vas a hacer tu primer cambio: Ana va a tener un pedido más, una **Mochila**.

Paso 7. Abrir principal.py para editarlo

1. En el Explorador de archivos, dentro de la carpeta **tienda**, haz **clic derecho** sobre `principal.py`.
2. Elige **Abrir con** y después **Bloc de notas**. (Si usas otro editor, como Visual Studio Code, también sirve).
3. Se abrirá el código del programa.

> No hagas doble clic sobre el archivo: eso lo ejecutaría en lugar de abrirlo para editar.

Paso 8. Escribir la línea nueva

Busca la parte del código que empieza con el comentario `# 1. Crear objetos`. Ahora mismo está así:

ANTES

```
    # 1. Crear objetos
    ana = Cliente("Ana")
    ana.pedidos.append(Pedido("Libro"))
    ana.pedidos.append(Pedido("Lápiz"))
```

Haz clic al final de la línea del **Lápiz**, pulsa **Enter** para crear una línea nueva y escribe la línea marcada en verde. Tiene que quedar así:

DESPUÉS

```
    # 1. Crear objetos
    ana = Cliente("Ana")
    ana.pedidos.append(Pedido("Libro"))
    ana.pedidos.append(Pedido("Lápiz"))
    ana.pedidos.append(Pedido("Mochila"))
```

Qué significa la línea nueva, de derecha a izquierda:

- `Pedido("Mochila")` crea un objeto nuevo de la clase **Pedido** cuyo producto es Mochila.
- `ana.pedidos` es la lista de pedidos de Ana.
- `.append(...)` añade ese pedido al final de la lista.

Fíjate en que la línea empieza con **cuatro espacios**, igual que la de arriba. En Python los espacios del principio son importantes: indican que la línea está dentro del bloque `with`.

Paso 9. Guardar y ejecutar

1. Guarda el archivo: menú **Archivo** y después **Guardar** (o **Ctrl + G** en el Bloc de notas).
2. Vuelve a la terminal.
3. Escribe `python principal.py` y pulsa **Enter**.

Ahora deben aparecer **tres** líneas, porque Ana tiene tres pedidos:

```
Libro es de Ana
Lápiz es de Ana
Mochila es de Ana
```

```
^Chiro@hiro-Lenovo-ideapad-330S-14IKB:/var/www/html/@CEAC/2DAM/Acceso a Datos/Ej
ercicios/015-Mapeo Python 3-11-2026/tienda$ python3 principal.py
Libro es de Ana
Lápiz es de Ana
Mochila es de Ana
```

PANTALLAZO 6

Pega aquí: la terminal mostrando los tres pedidos de Ana.

4. Parte 3: crear al cliente Luis

Ahora vas a añadir un cliente nuevo, **Luis**, que ha pedido un **Cuaderno** y una **Goma**. Lo harás igual que se hizo con Ana.

Paso 10. Crear a Luis y sus pedidos

Vuelve al Bloc de notas donde tienes abierto `principal.py`. Debajo de la línea de la Mochila deja una línea en blanco y escribe las tres líneas marcadas en verde:

```
    # 1. Crear objetos
    ana = Cliente("Ana")
    ana.pedidos.append(Pedido("Libro"))
    ana.pedidos.append(Pedido("Lápiz"))
    ana.pedidos.append(Pedido("Mochila"))

    luis = Cliente("Luis")
    luis.pedidos.append(Pedido("Cuaderno"))
    luis.pedidos.append(Pedido("Goma"))
```

Qué hace cada línea:

- `luis = Cliente("Luis")` crea un objeto nuevo de la clase **Cliente** con el nombre Luis y lo guarda en la variable `luis`.
- `luis.pedidos.append(Pedido("Cuaderno"))` crea un pedido de un Cuaderno y lo añade a los pedidos de Luis.
- `luis.pedidos.append(Pedido("Goma"))` hace lo mismo con la Goma.

Recuerda poner **cuatro espacios** al principio de cada línea.

Paso 11. Guardar a Luis en la base de datos

Crear el objeto no es suficiente: hay que decirle a la sesión que también lo guarde. Busca la parte `# 2. Guardar`. Ahora mismo está así:

ANTES

```
    # 2. Guardar
    sesion.add(ana)
    sesion.commit()
```

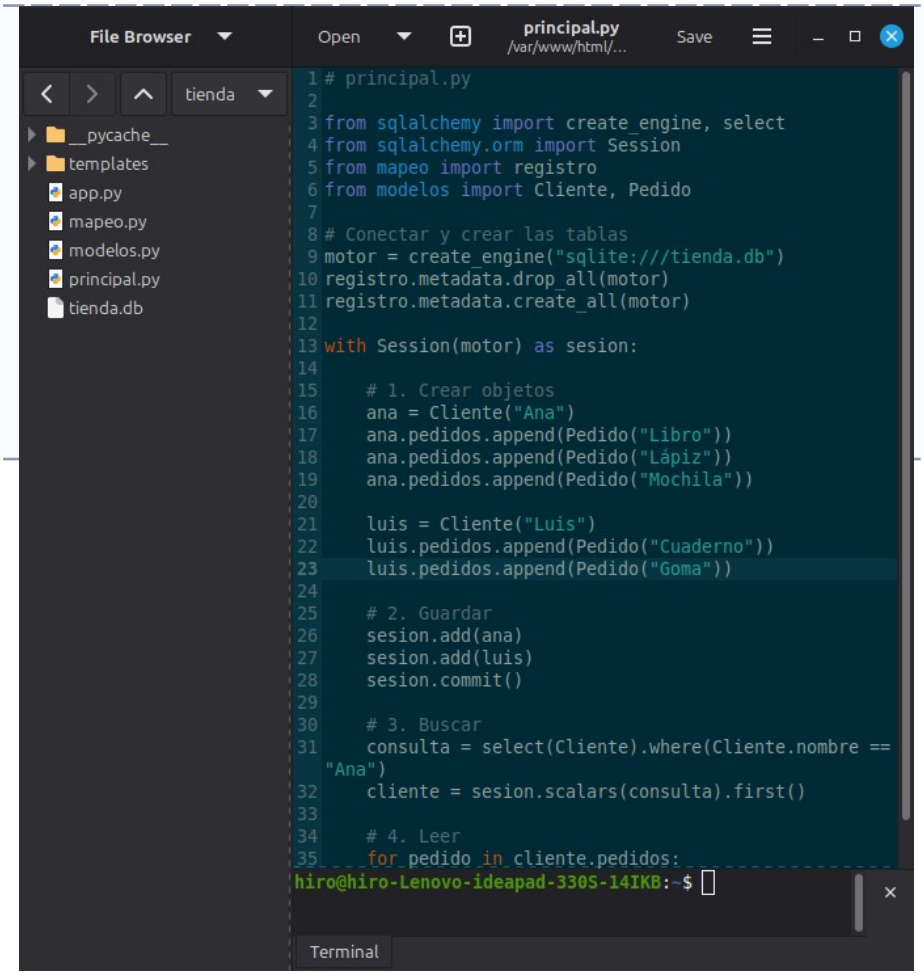
Haz clic al final de la línea `sesion.add(ana)`, pulsa **Enter** y escribe la línea marcada en verde:

DESPUÉS

```
    # 2. Guardar
    sesion.add(ana)
    sesion.add(luis)
    sesion.commit()
```

- `sesion.add(luis)` le dice al ORM que Luis también hay que guardarlo.
- Sus pedidos se guardan solos, igual que los de Ana, gracias a la relación que está escrita en `mapeo.py`.
- `sesion.commit()` guarda de verdad todo en la base de datos. Por eso la línea nueva va **antes** del `commit`.

Guarda el archivo (**Archivo** y **Guardar**).



PANTALLAZO 7

Pega aquí: el Bloc de notas con principal.py, donde se vean las líneas nuevas de Luis.

5. Parte 4: buscar a Luis

El programa, al final, busca a una clienta por su nombre y escribe sus pedidos. Ahora mismo busca a Ana. Vas a cambiarlo para que busque a **Luis**.

Paso 12. Cambiar el nombre en la búsqueda

Busca la parte `# 3. Buscar`. Ahora mismo está así:

ANTES

```
    # 3. Buscar
    consulta = select(Cliente).where(Cliente.nombre == "Ana")
```

Cambia solo la palabra **Ana** por **Luis** (dentro de las comillas). Tiene que quedar así:

DESPUÉS

```
    # 3. Buscar
    consulta = select(Cliente).where(Cliente.nombre == "Luis")
```

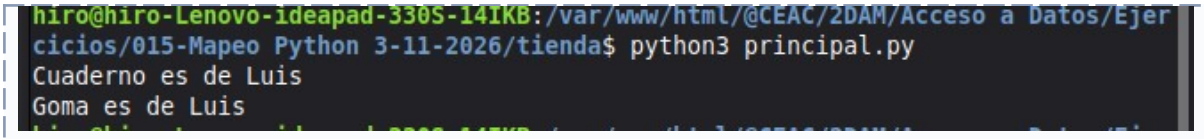
Esta línea significa: **selecciona los clientes cuyo nombre sea Luis**. El ORM la traduce a SQL por ti.

Paso 13. Guardar y ejecutar

1. Guarda el archivo (**Archivo** y **Guardar**).
2. Vuelve a la terminal.
3. Escribe `python principal.py` y pulsa **Enter**.

Ahora deben aparecer los pedidos de Luis:

```
Cuaderno es de Luis
Goma es de Luis
```



PANTALLAZO 8

Pega aquí: la terminal mostrando los dos pedidos de Luis.

6. Parte 5: cambiar el título de la web

Por último vas a cambiar el título que aparece arriba en la página web.

Paso 14. Abrir index.html para editarlo

1. En el Explorador de archivos, entra en la carpeta `templates`.
2. Haz **clic derecho** sobre `index.html`, elige **Abrir con** y después **Bloc de notas**.

> No hagas doble clic: se abriría en el navegador y no podrías editarlo.

Paso 15. Cambiar el título

Busca esta línea:

ANTES

```
  <h1>Clientes</h1>
```

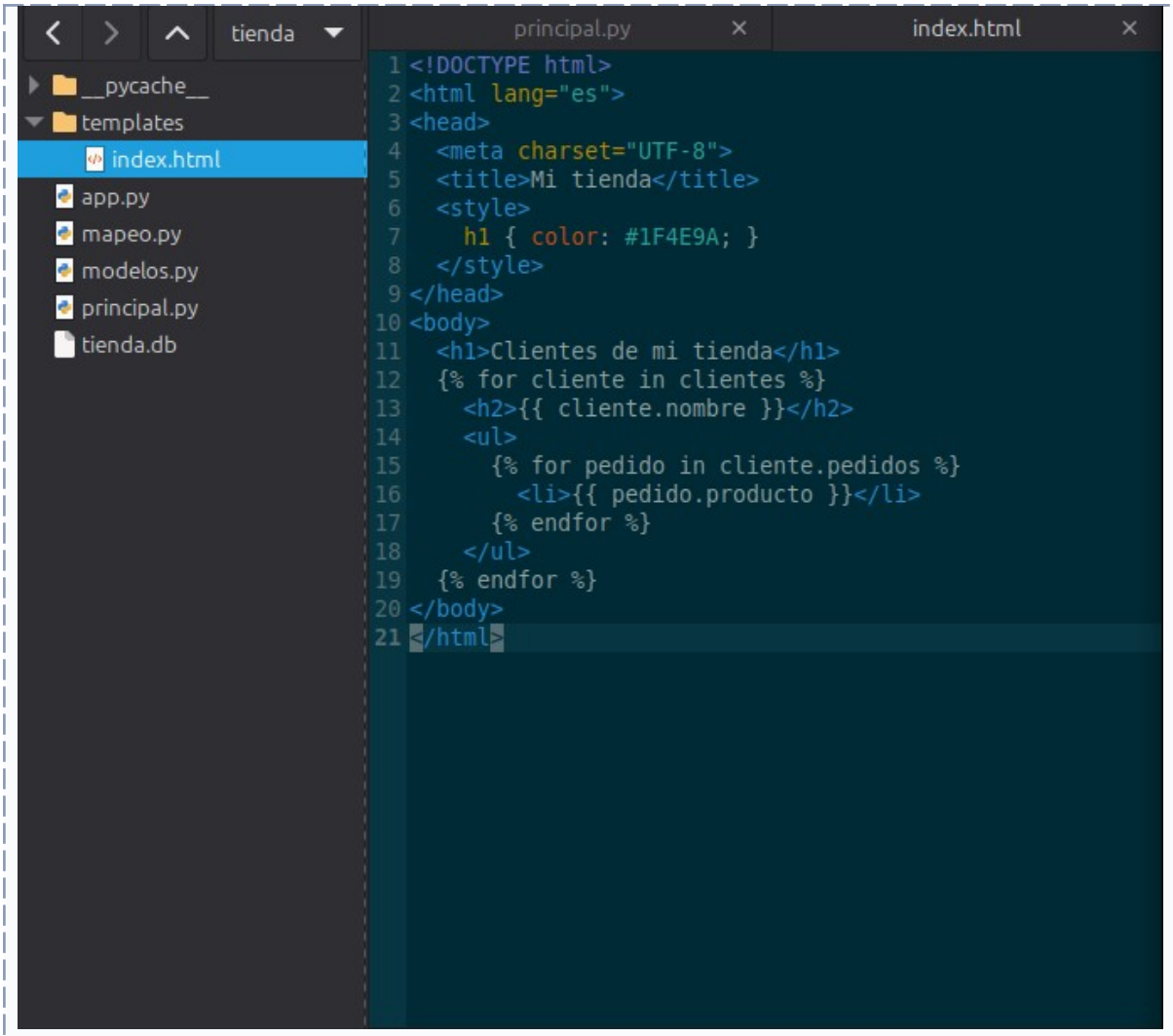
Cambia la palabra **Clientes** por **Clientes de mi tienda**, sin tocar las etiquetas `<h1>` y `</h1>`:

DESPUÉS

```
  <h1>Clientes de mi tienda</h1>
```

La etiqueta `<h1>` sirve para poner el título principal de una página. Lo que hay entre `<h1>` y `</h1>` es el texto que se ve.

Guarda el archivo (**Archivo** y **Guardar**).



PANTALLAZO 9

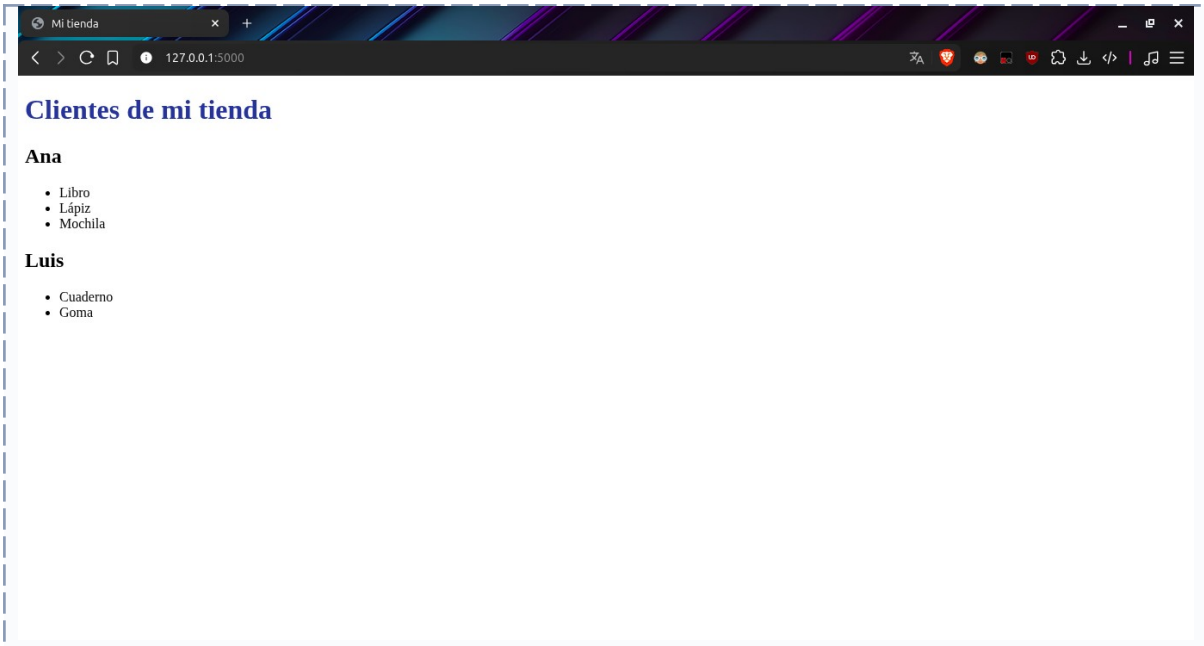
Pega aquí: el Bloc de notas con index.html y el título cambiado.

Paso 16. Ver el resultado final

1. Vuelve a la terminal.
2. Escribe `python app.py` y pulsa **Enter**.
3. Espera a que aparezca `* Running on http://127.0.0.1:5000`.
4. Ve al navegador y escribe `http://127.0.0.1:5000` en la barra de direcciones. Si ya tenías la página abierta, pulsa **F5** para recargarla.

Ahora la web debe mostrar el título **Clientes de mi tienda** y debajo a los dos clientes:

- **Ana**, con Libro, Lápiz y Mochila.
- **Luis**, con Cuaderno y Goma.



PANTALLAZO 10

Pega aquí: el navegador mostrando la web final con Ana y Luis.

Paso 17. Terminar

1. Haz clic en la terminal y pulsa **Ctrl + C** para parar el servidor.
2. Cierra la terminal y el Bloc de notas.

¡Ejercicio terminado! Has añadido objetos nuevos en Python y el ORM los ha guardado en la base de datos sin que tuvieras que escribir nada de SQL.